Agentic Specification Refinement

Mid-Submission Progress Report - Team ReqHQ

Course	CS6.403 Autonomous Software Engineering, IIIT Hyderabad, Monsoon 2026
Team	Panem Chaitanya Pavan Kumar (2025204023) · Anjali Yadav (2025204015) · Anshul Agrawal (2025901015)
Date	6 November 2026
Repository	`github.com/panemchaitanya/Agentic-Specification-Refinement`
Hosted demo	`agentic-specification-refinement.onrender.com`

1. Project overview

Product requirement documents (PRDs) are usually strong on what a system should do and weak on how well it should do it: performance budgets, security expectations and reliability targets are often missing, vague, or scattered across long documents. Writing a complete specification from such a PRD is slow, and a single-pass LLM draft inherits the same gaps. Our project builds an agentic pipeline that turns a rough PRD into a complete, *trace-linked* specification: every line in the final spec names where it came from - a verbatim PRD sentence, a specialist critic, or a human answer.

The system has four cooperating components:

- **Orchestrator.** Reads the PRD and drafts functional requirements, each traced to the exact PRD sentence it came from.
- **Specialist critics.** Three reviewers - Security, Architecture and Performance - inspect the draft and the PRD for missing or vague non-functional requirements. Each finding quotes the PRD sentence that triggered it and explains why it matters.
- **Gap-and-conflict check.** Applies the escalation rule from our proposal: the human is asked only when a critic reports missing business context or two critics disagree. Vague wording a critic can fix itself is repaired directly and visibly marked as not user-confirmed.
- **Trace-linked composer.** Produces the final specification grouped into functional requirements and NFR categories, downloadable as JSON or Markdown, with a trace report counting how every line is grounded.

Later stages train small specialist models (SLMs) to replace the LLM critics, and evaluate the whole system on frozen dataset splits against these hypotheses.

2. Research hypotheses (verbatim from the submitted proposal)

> **H1.** The full pipeline (without human input) has higher NFR recall than a single LLM given the same PRD, with precision dropping by no more than 0.03.

> **H2.** SLM critics come within 0.05 NFR F1 of LLM critics in the same roles, at much lower token cost and latency.

> **H3.** Targeted questioning reaches at least 95% of the completeness of asking the human to review everything, using at least 50% fewer questions.

No hypothesis results are claimed at this checkpoint. The evaluation harness, frozen splits and gold protocol needed to test H1-H3 exist and are described in Section 3; scoring runs only after the gold set is fully human-verified.

3. Work completed

3.1 Stage 1 - Working PRD-to-spec pipeline

The full loop from raw PRD text to a downloadable, trace-linked specification works end to end. The code is organized as a small, testable core (`src/specpipe/`: schemas, LLM client, orchestrator, critics, gap check, pipeline) behind a Streamlit UI. A deterministic mock model lets the whole application run and be tested offline; without an API key the app runs in a clearly labelled demo mode used for development and tests. The trained classifier is not yet integrated, so the hosted demo shows the flow with demo-mode output, not model quality; demo-mode output is never used for evaluation.

- Orchestrator drafts FRs, each traced to a verbatim PRD sentence.
- Security / Architecture / Performance critics flag missing or vague NFRs with the triggering PRD sentence and rationale.
- Gap check deduplicates findings, applies the escalation rule, and asks at most 5 targeted questions; unanswered questions enter the spec as open items instead of being silently resolved.
- Final spec marks every line's source; lines the critic repaired without asking are marked `user_confirmed: false` and counted separately in the trace report, so visibility is never confused with approval.
- 87 offline unit tests cover the pipeline contracts; the hosted demo (link on page 1) deploys manually so it stays stable during testing windows.

3.2 User interface

A guided three-step wizard: (1) **Describe** - paste a PRD or load a bundled sample and analyse it; (2) **Review & answer** - inspect draft FRs with their PRD traces and critic findings, answering only the questions the system cannot settle; (3) **Specification** - the final trace-linked spec, downloadable as JSON or Markdown.

3.3 Stage 2 - Frozen datasets and the gold protocol

Two datasets are frozen under `data/frozen/` with manifests, provenance and license documentation: PROMISE NFR (classification subtask) and PURE (61 requirement documents for the end-to-end evaluation). The dev/test split is fixed and the test split stays untouched until the Stage 4 evaluation, so no iterative tuning can leak into final numbers.

Because PURE ships without per-requirement NFR gold labels, we built a two-step ground-truth protocol:

- **LLM-suggested labels.** A free-tier model proposed requirement labels for the 61 dev documents, recorded with status "LLM-SUGGESTED - unverified, not gold". The evaluation loader rejects these fail-closed: unverified suggestions can never be scored as gold.
- **Anchored human verification.** Two annotators verify the suggestions with fixed written guidelines and a paging review tool that records exactly how many characters were presented and read. Documents are split into two unique parts plus a shared subsample reviewed independently by both; anchored accept/reject agreement (Cohen's kappa) is computed on the shared docs at merge time.
- **Fail-closed merge.** A document is promoted to FROZEN gold only when every suggestion has a verified resolution and the annotator confirmed reading the full scope. Any skip, tie, contested edit or incomplete read makes the whole document review-incomplete and ineligible for scoring - there is no partial-gold scoring.

3.4 Gold verification status (6 Nov 2026)

The human verification pass is underway. The first annotator's verdicts arrived on 6 November and were validated against the protocol (verdict coverage of every suggestion, full-scope reading confirmation, assignment membership). Current state:

Document	Suggestions	Verdict state	Eligible for gold
`pure-0000 cctns`	28	All resolved (27 accept incl. 2 edits, 1 reject, 1 human-added requirement)	Yes - FROZEN 6 Nov
`pure-0000 gamma_j`	25	All resolved (25 accept, 1 edit)	Yes - FROZEN 6 Nov
`pure-0000 inventory`	23	22 resolved, 1 skipped	No - awaits adjudication of the skip
`pure-2001 ctc_network`	40	All resolved, but full-scope read not completed	No - needs full-scope re-review
`pure-2000 nasa_x38`, `pure-2001 hats`	40 / 26	All suggestions skipped	No - review pending

These verdicts are merged into the repository: 2 documents are FROZEN (single annotator, so no inter-annotator kappa is recorded yet) and the evaluation loader reports 2 eligible / 59 excluded, the rest refused fail-closed. Scoring the eligible pair is currently blocked on the free-tier model's exhausted daily request quota (the rubric-matching step needs the model), not on the remaining verdicts; we make no gold-based metric claims in this report. Remaining verdict coverage is the honest bottleneck of this stage: process-limited, not design-limited.

3.5 Stage 3 - Training pilot (feasibility)

We trained the NFR classification subtask on the frozen PROMISE dev split with a stratified dev-internal holdout, to measure feasibility and the honest floor a trained specialist must beat:

Model / setting	Accuracy	Macro-F1	Notes
TF-IDF + logistic regression	0.744	0.679	Classical baseline
DistilBERT, CPU pilot	0.779	0.504	Memory-bound pilot
DistilBERT, GPU sweep best (6 ep, batch 16, lr 5e-5)	0.826	0.782	Colab T4, 21 s, 1.5 GB peak; historical run, checkpoint not preserved
Weakest sweep config	0.523	0.235	Kept: shows lr/batch sensitivity

Training is currently unseeded: the same best config measured macro-F1 0.782 (GPU sweep run, not preserved), 0.7571 (first preservation run; its checkpoint was lost when the Colab VM was recycled) and 0.6489 (the retrain whose checkpoint is actually preserved) across three runs of the same frozen dev holdout. The 0.782 and 0.7571 numbers are historical, unpreserved pilot runs; the only checkpoint we hold today is the 0.6489 retrain, kept byte-verified outside git with hashes and assembly instructions. Sweep numbers are recorded as feasibility evidence, not as achieved-by-claim for the preserved checkpoint. Seeded, reproducible training is scheduled follow-up work.

3.6 Stage 4 - Evaluation harness and pre-freeze generation

The evaluation harness is built and documented (`docs/evaluation.md`, decisions D1-D4): a frozen-split runner with baselines B1-B4, gold loading that refuses unverified documents, a matching rubric, and per-run JSONL logs. On 6 November we ran bounded pre-freeze **generation** over the dev split: 43 of 61 documents now hold complete B1+B2+B3 output triplets with persisted requirements, bounded to the same 120k-character prefix scope the gold protocol uses. The run stopped cleanly on the free-tier daily request quota with 18 documents not yet generated. These outputs are **unscored**: scoring runs only after the gold freeze, and only the complete triplets form the comparable subset. No evaluation metric is claimed in this report.

4. Known gaps, carried honestly

- **Gold coverage is scope-bounded.** The LLM pre-labeler and human review see at most the first 120,000 characters per document (18 of 61 dev documents are longer); dev scoring uses the same bound and no full-document recall is claimed.
- **FR/NFR typing is a keyword heuristic** in the orchestrator, unvalidated; the Stage 2 guidelines and matching rubric own the real boundary.
- **Unseeded training** makes classifier quality vary run to run (Section 3.5); seeded training and a reproducibility check are open work.
- **"0 untraced lines" means trace coverage only** - every line names a source; it says nothing about correctness, which is exactly what the frozen-split evaluation will measure.
- **Evaluation is not scored yet.** Generation outputs exist for 43/61 dev documents; gold freeze and rubric scoring are the immediate next steps.

5. Milestones achieved vs. plan

Planned stage	Status at mid-submission
Stage 1: pipeline + UI + hosted demo	**Core complete.** End-to-end loop, 3-step wizard, 87 offline tests, public demo. The hosted app runs the deterministic demo mode only; the trained model is not yet integrated, so the demo performs no real model inference.
Stage 2: dataset freeze + gold protocol	**Datasets complete; gold in progress.** Splits frozen with provenance; verification tooling, guidelines and assignments done; first 2 documents FROZEN on 6 Nov; rest in review.
Stage 3: SLM training pilot	**Feasibility shown.** Best sweep 0.826 acc / 0.782 macro-F1; checkpoint preserved; seeded training pending.
Stage 4: frozen evaluation of H1-H3	**Bounded-dev harness implemented; final protocol not yet executed.** 43/61 dev generation triplets ready; 2 documents FROZEN; rubric scoring currently blocked on the free-tier daily quota.

6. Next steps

- Complete gold adjudication (merge the verified verdicts, re-review the blocked documents, collect the second annotator's shared docs for kappa), then run the bounded dev evaluation and rubric matching on the eligible subset.
- H2 treatment arm: fine-tune `google/flan-t5-base` on the frozen train split and integrate it as a third critic behind a flag; integrate the trained DistilBERT tagger into the UI for kind-specific critic routing.
- Seeded classifier training with a reproducibility check.
- Full H1-H3 comparison on the untouched TEST split, once, under the fixed protocol.

7. Repository and development history

Development is tracked in the GitHub repository linked on page 1 (active development branch: `Chaitanya`). The commit history records the work above as it happened, including: the initial pipeline and UI build with a deterministic demo mode; the dataset freeze with leak checks and provenance documentation; the training pilot and checkpoint preservation with honest run identity; the evaluation harness with fail-closed gold loading; the bounded generation driver with rate-limit-aware retries and per-document run logs, and the first anchored human-verified gold merge. The current tree documents its own state: the README carries the same milestone status, known gaps and backlog as this report.

All evaluation scripts, frozen manifests, annotation guidelines and run logs referenced in this report are in the repository, and the trained checkpoint is preserved with byte-level hashes for verification.